Django Backend Development & Deployment — Full Notes

Python + Django + DRF + Redis + Celery + PostgreSQL + Gunicorn + Nginx

---

01. WSL2 and Ubuntu Setup

What is WSL?

WSL (Windows Subsystem for Linux) allows you to run a Linux environment inside Windows. It is useful for practicing Linux-based Django deployment locally.

Install WSL2

Open PowerShell as Administrator:

```
wsl --install
```

Restart your computer if required. Open Ubuntu and create a Linux username and password.

Check WSL:

```
wsl --status
wsl --list --verbose
```

Update Ubuntu:

```
sudo apt update
sudo apt upgrade -y
```

Install essential tools:

```
sudo apt install -y python3 python3-pip python3-venv python3-dev \
build-essential git curl nginx
```

Check versions:

```
python3 --version
git --version
nginx -v
```

Important:

- WSL2 is useful for practicing Linux deployment on Windows.

- Production Django applications are commonly deployed on Linux servers or VPS instances.
- Windows Python and WSL Python environments are separate.
- Store Linux projects inside the WSL filesystem, such as `~/projects/`, for better performance.

---

02. Linux Terminal Basics

Print the current directory:

```
pwd
```

List files:

```
ls
ls -la
```

Create a directory:

```
mkdir projects
```

Navigate into a directory:

```
cd projects
```

Go to the parent directory:

```
cd ..
```

Create a file:

```
touch example.txt
```

Read a file:

```
cat example.txt
```

Edit a file:

```
nano example.txt
```

Copy files:

```
cp source.txt copy.txt
cp -r source_folder copy_folder
```

Move or rename files:

```
mv old.txt new.txt
```

Delete a file:

```
rm example.txt
```

Delete a directory:

```
rm -r folder_name
```

Warning: Use `rm -rf` carefully. An incorrect path can permanently delete important files.

---

03. Python Virtual Environment

Create a project directory

```
mkdir -p ~/projects/django-deployment
cd ~/projects/django-deployment
```

Create a virtual environment

```
python3 -m venv venv
```

Activate it:

```
source venv/bin/activate
```

You should see `(venv)` in your terminal.

Upgrade pip:

```
python -m pip install --upgrade pip
```

Install Django and Django REST Framework:

```
pip install django djangorestframework
```

List installed packages:

```
pip list
```

Save dependencies:

```
pip freeze > requirements.txt
```

Install dependencies in another environment:

```
pip install -r requirements.txt
```

Deactivate the environment:

```
deactivate
```

Create a `.gitignore` file

```
venv/
.venv/
__pycache__/
*.py[cod]
.env
db.sqlite3
staticfiles/
media/
.pytest_cache/
.coverage
```

Never commit secrets or sensitive environment files to GitHub. Back up production database data and user-uploaded media separately.

---

04. Django Project Setup

Install Django:

```
pip install django
```

Create a project:

```
django-admin startproject config .
```

Create an application:

```
python manage.py startapp core
```

Project structure:

```
django-deployment/
├── config/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
```

```
├── core/
│   ├── migrations/
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
├── manage.py
├── requirements.txt
└── .gitignore
```

Register the application

In `config/settings.py`:

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    "rest_framework",

    "core",
]
```

Apply migrations

```
python manage.py migrate
```

Create an administrator:

```
python manage.py createsuperuser
```

Run the development server:

```
python manage.py runserver
```

Open:

```
http://127.0.0.1:8000/
```

Important: Django's `runserver` is for development, not public production deployment.

---

05. Django Fundamentals

Create a model

In `core/models.py`:

```
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=200)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

Create and apply migrations:

```
python manage.py makemigrations
python manage.py migrate
```

Register the model in Django Admin

In `core/admin.py`:

```
from django.contrib import admin
from .models import Product

admin.site.register(Product)
```

Create a view

In `core/views.py`:

```
from django.http import JsonResponse


def health_check(request):
    return JsonResponse({"status": "ok"})
```

Create `core/urls.py`:

```
from django.urls import path
from .views import health_check

urlpatterns = [
```

```
    path("health/", health_check, name="health_check"),
]
```

Update `config/urls.py`:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/", include("core.urls")),
]
```

Test the endpoint:

```
http://127.0.0.1:8000/api/health/
```

Expected response:

```
{
  "status": "ok"
}
```

---

06. Django REST Framework (DRF)

Install DRF:

```
pip install djangorestframework
```

Add `"rest_framework"` to `INSTALLED_APPS`.

Create a serializer

Create `core/serializers.py`:

```
from rest_framework import serializers
from .models import Product


class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = ["id", "name", "price", "created_at"]
        read_only_fields = ["id", "created_at"]
```

Create API views

In `core/views.py`:

```
from rest_framework import generics
from .models import Product
from .serializers import ProductSerializer


class ProductListCreateAPIView(generics.ListCreateAPIView):
    queryset = Product.objects.all().order_by("-id")
    serializer_class = ProductSerializer


class ProductDetailAPIView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
```

Update `core/urls.py`:

```
from django.urls import path
from .views import (
    health_check,
    ProductListCreateAPIView,
    ProductDetailAPIView,
)

urlpatterns = [
    path("health/", health_check, name="health_check"),
    path("products/", ProductListCreateAPIView.as_view(), name="product-
list"),
    path(
        "products/<int:pk>/",
        ProductDetailAPIView.as_view(),
        name="product-detail",
    ),
]
```

API endpoints

Method	Endpoint	Purpose
GET	`/api/products/`	List products

POST	`/api/products/`	Create a product
GET	`/api/products/1/`	Retrieve a product
PUT/PATCH	`/api/products/1/`	Update a product
DELETE	`/api/products/1/`	Delete a product

Example POST body:

```
{
  "name": "Keyboard",
  "price": "1200.00"
}
```

For production APIs, configure authentication, permissions, pagination, throttling, and appropriate validation.

---

07. Redis Installation and Basic Commands

What is Redis?

Redis is an in-memory data store commonly used for:

- Caching
- Celery message brokering
- Celery result storage
- Django Channels channel layers
- Some types of session storage

Redis does not replace a relational database for typical transactional application data. PostgreSQL is a common choice for persistent relational data.

Install Redis on Ubuntu

```
sudo apt update
sudo apt install -y redis-server
```

Check the service:

```
sudo systemctl status redis-server
```

Open the Redis CLI:

```
redis-cli
```

Run these commands:

```
PING
SET name "Faruk"
GET name
DEL name
EXIT
```

Expected response to `PING`:

```
PONG
```

Check Redis directly:

```
redis-cli ping
```

Install the Python Redis client

Activate your virtual environment:

```
pip install redis
```

Test Redis with Python:

```
import redis

client = redis.Redis(
    host="127.0.0.1",
    port=6379,
    db=0,
    decode_responses=True,
)

client.set("message", "Hello Redis")
print(client.get("message"))
```

Default Redis port:

```
6379
```

Security: Do not expose Redis directly to the public internet. Configure network access, authentication, firewall rules, and appropriate binding.

---

08. Django Redis Cache

Caching can reduce repeated database queries and expensive computations.

Install django-redis

```
pip install django-redis
```

Configure `config/settings.py`:

```
CACHES = {
    "default": {
        "BACKEND": "django_redis.cache.RedisCache",
        "LOCATION": "redis://127.0.0.1:6379/1",
        "OPTIONS": {
            "CLIENT_CLASS": "django_redis.client.DefaultClient",
        },
        "TIMEOUT": 300,
    }
}
```

This example uses Redis database `1` for caching.

Create a cached view

In `core/views.py`:

```
from django.core.cache import cache
from django.http import JsonResponse


def cached_message(request):
    message = cache.get("welcome_message")

    if message is None:
        message = "Hello from Django and Redis"
        cache.set("welcome_message", message, timeout=300)

    return JsonResponse({"message": message})
```

Add the URL:

```
path("cache/", cached_message, name="cached-message"),
```

Cache API

```
from django.core.cache import cache

cache.set("username", "faruk", timeout=60)

username = cache.get("username")

cache.delete("username")
```

`cache.clear()` removes all keys in the configured cache and can affect other applications sharing it. Avoid using it on shared production Redis instances unless that behavior is intentional.

Cache invalidation is important. When a product changes, update or invalidate the corresponding cached data.

---

09. Django Sessions with Redis

Sessions store user-specific state, such as login session data or temporary workflow information.

Django's default session backend uses the database. To use cache-backed sessions, configure the backend appropriately.

Install:

```
pip install django-redis
```

For cache-backed sessions, add to `settings.py`:

```
SESSION_ENGINE = "django.contrib.sessions.backends.cache"
SESSION_CACHE_ALIAS = "default"
```

Cache-backed sessions can be lost if the cache is cleared or its data is evicted.

For sessions stored in both the cache and database:

```
SESSION_ENGINE = "django.contrib.sessions.backends.cached_db"
SESSION_CACHE_ALIAS = "default"
```

Apply migrations if necessary:

```
python manage.py migrate
```

Session example:

```
request.session["cart_count"] = 3

cart_count = request.session.get("cart_count", 0)
```

For production, configure HTTPS, secure cookies, session expiry, and `SESSION_COOKIE_SECURE` appropriately.

---

10. Celery Setup with Redis

What is Celery?

Celery is a distributed task queue that runs time-consuming tasks outside the normal web request cycle.

Common use cases:

- Sending emails
- Generating reports
- Processing large files
- Synchronizing external APIs
- Running scheduled tasks

Redis can act as Celery's message broker.

Install Celery

```
pip install celery redis
```

Create `config/celery.py`:

```
import os

from celery import Celery

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")

app = Celery("config")

app.config_from_object("django.conf:settings", namespace="CELERY")
app.autodiscover_tasks()
```

Update `config/__init__.py`:

```
from .celery import app as celery_app

__all__ = ("celery_app",)
```

Configure Celery in `settings.py`:

```
CELERY_BROKER_URL = "redis://127.0.0.1:6379/2"
CELERY_RESULT_BACKEND = "redis://127.0.0.1:6379/3"

CELERY_ACCEPT_CONTENT = ["json"]
CELERY_TASK_SERIALIZER = "json"
CELERY_RESULT_SERIALIZER = "json"
CELERY_TIMEZONE = "Asia/Dhaka"
```

In production, load Redis URLs and credentials from protected environment variables.

---

11. Create and Run Celery Tasks

Create `core/tasks.py`:

```
import time

from celery import shared_task


@shared_task
def add_numbers(a, b):
    return a + b


@shared_task
def generate_report():
    time.sleep(5)
    return "Report generated"
```

Terminal 1: Start Redis

```
sudo systemctl start redis-server
```

Terminal 2: Run Django

```
python manage.py runserver
```

Terminal 3: Start the Celery worker

From the project root with the virtual environment active:

```
celery -A config worker --loglevel=info
```

Trigger a task

Open the Django shell:

```
python manage.py shell
```

```
from core.tasks import add_numbers

result = add_numbers.delay(10, 20)
print(result.id)
```

The task is queued and executed by the worker.

Retrieve the result:

```
print(result.get(timeout=10))
```

Avoid blocking web requests while waiting for Celery results. The purpose of asynchronous tasks is to let the request finish without waiting for long-running work.

---

12. Celery Beat — Scheduled Tasks

Celery Beat schedules tasks and sends them to the queue. A Celery worker executes them.

Install Django Celery Beat:

```
pip install django-celery-beat
```

Add to `INSTALLED_APPS`:

```
INSTALLED_APPS = [
    # Existing apps
    "django_celery_beat",
]
```

Run migrations:

```
python manage.py migrate
```

Example schedule in `settings.py`:

```
from celery.schedules import crontab

CELERY_BEAT_SCHEDULE = {
    "generate-report-every-day": {
        "task": "core.tasks.generate_report",
```

```
        "schedule": crontab(hour=2, minute=0),
    },
}
```

This schedules the task daily at 2:00 AM, provided the worker and Beat are running and timezone settings are correct.

Run Beat:

```
celery -A config beat --loglevel=info
```

Run the worker in another terminal:

```
celery -A config worker --loglevel=info
```

Usually, only one active Beat scheduler should manage a given schedule. Multiple schedulers can produce duplicate tasks.

---

13. PostgreSQL Setup

PostgreSQL is a common relational database choice for production Django applications.

Install PostgreSQL

```
sudo apt update
sudo apt install -y postgresql postgresql-contrib libpq-dev
```

Check the service:

```
sudo systemctl status postgresql
```

Open PostgreSQL:

```
sudo -u postgres psql
```

Create a database and user:

```
CREATE DATABASE django_db;
CREATE USER django_user WITH PASSWORD 'CHANGE_THIS_PASSWORD';

ALTER ROLE django_user SET client_encoding TO 'utf8';
ALTER ROLE django_user SET default_transaction_isolation TO 'read committed';
ALTER ROLE django_user SET timezone TO 'UTC';

GRANT ALL PRIVILEGES ON DATABASE django_db TO django_user;
```

Exit:

```
\q
```

Use a strong password in production. Depending on the PostgreSQL version and permission model, additional schema privileges may be required.

Install the Python driver

```
pip install "psycopg[binary]"
```

Configure Django

In `settings.py`:

```
import os

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.environ["DB_NAME"],
        "USER": os.environ["DB_USER"],
        "PASSWORD": os.environ["DB_PASSWORD"],
        "HOST": os.environ.get("DB_HOST", "127.0.0.1"),
        "PORT": os.environ.get("DB_PORT", "5432"),
    }
}
```

After setting the environment variables:

```
python manage.py migrate
```

Do not expose PostgreSQL directly to the public internet. Allow access only from trusted application servers or networks.

---

14. Production Settings and Environment Variables

Production configuration should not rely on development defaults.

Example environment variables

```
DJANGO_SECRET_KEY=replace-with-a-long-random-secret
DJANGO_DEBUG=False
DJANGO_ALLOWED_HOSTS=example.com,www.example.com

DB_NAME=django_db
DB_USER=django_user
```

```
DB_PASSWORD=replace-with-a-strong-password
DB_HOST=127.0.0.1
DB_PORT=5432
```

These are examples only. Never commit actual secrets to a public repository.

Configure `settings.py`

```
import os

SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]

DEBUG = os.environ.get("DJANGO_DEBUG", "False").lower() == "true"

ALLOWED_HOSTS = [
    host.strip()
    for host in os.environ.get("DJANGO_ALLOWED_HOSTS", "").split(",")
    if host.strip()
]
```

Keep `DEBUG=False` in production. Add the real domain names or hostnames to `ALLOWED_HOSTS`.

After configuring HTTPS and the reverse proxy, enable appropriate settings:

```
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SECURE_SSL_REDIRECT = True
```

Enable HTTPS redirects only after verifying that HTTPS and proxy configuration work correctly. Otherwise, redirect loops or inaccessible pages may occur.

Additional security settings:

```
SECURE_CONTENT_TYPE_NOSNIFF = True
X_FRAME_OPTIONS = "DENY"
```

Enable HSTS only after HTTPS is fully verified. Incorrect HSTS settings can cause persistent browser access problems.

Run the deployment checks

```
python manage.py check --deploy
```

Review and resolve the reported warnings.

---

15. Gunicorn Setup

What is Gunicorn?

Gunicorn is a WSGI HTTP server that can serve Django's WSGI application.

Typical architecture:

```
Browser
   |
   v
Nginx
   |
   v
Gunicorn
   |
   v
Django WSGI
   |
   v
PostgreSQL / Redis
```

Install Gunicorn

```
pip install gunicorn
```

Run it from the Django project root:

```
gunicorn config.wsgi:application --bind 127.0.0.1:8000
```

Open:

```
http://127.0.0.1:8000/
```

Stop the server with `Ctrl + C`.

Example with multiple workers:

```
gunicorn config.wsgi:application \
    --bind 127.0.0.1:8000 \
    --workers 3 \
    --timeout 60
```

The worker count is only an example. Choose it according to available CPU, memory, and application workload. Too many workers can exhaust memory.

WSGI vs ASGI

- `config.wsgi:application`: traditional synchronous Django deployment.
- `config.asgi:application`: ASGI-compatible application, useful for asynchronous protocols and WebSockets.
- Gunicorn commonly serves Django WSGI applications.
- ASGI deployments can use an ASGI server such as Uvicorn or Daphne, with a suitable process manager if required.

---

16. Nginx Reverse Proxy and Static Files

What is Nginx?

Nginx can receive public requests and forward them to Gunicorn. It can also serve static files and terminate HTTPS connections.

Configure static files

In `settings.py`:

```
STATIC_URL = "/static/"
STATIC_ROOT = BASE_DIR / "staticfiles"
```

Collect static files:

```
python manage.py collectstatic
```

Create an Nginx configuration

Assume:

- Domain: `example.com`
- Django application: `127.0.0.1:8000`
- Project path: `/home/djangoapp/django-deployment`

Create the file:

```
sudo nano /etc/nginx/sites-available/djangoapp
```

Configuration:

```
server {
    listen 80;
    server_name example.com www.example.com;

    client_max_body_size 20M;

    location /static/ {
```

```
        alias /home/djangoapp/django-deployment/staticfiles/;
    }

    location / {
        proxy_pass http://127.0.0.1:8000;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Change the domain and filesystem path to match your server. Ensure Nginx has permission to read the static files.

Enable the configuration:

```
sudo ln -s /etc/nginx/sites-available/djangoapp \
/etc/nginx/sites-enabled/djangoapp
```

If the default site conflicts with your configuration, disable it as appropriate:

```
sudo rm /etc/nginx/sites-enabled/default
```

Test the configuration:

```
sudo nginx -t
```

Reload Nginx:

```
sudo systemctl reload nginx
```

Check status:

```
sudo systemctl status nginx
```

Important notes

- Bind Gunicorn to `127.0.0.1` in a typical same-server deployment.
- Nginx should handle public HTTP/HTTPS traffic.
- Open ports 80 and 443 in your firewall as required.
- Check filesystem permissions if static files are unavailable.
- Plan secure storage and serving rules for user-uploaded media.

---

17. Gunicorn with systemd

`systemd` manages background services, restarts failed processes, and starts services during system boot.

Assume the project path is:

```
/home/djangoapp/django-deployment
```

Virtual environment:

```
/home/djangoapp/django-deployment/venv
```

Create the service file:

```
sudo nano /etc/systemd/system/gunicorn.service
```

Example:

```
[Unit]
Description=Gunicorn Django Application
After=network.target

[Service]
User=djangoapp
Group=www-data
WorkingDirectory=/home/djangoapp/django-deployment
ExecStart=/home/djangoapp/django-deployment/venv/bin/gunicorn \
    --workers 3 \
    --bind 127.0.0.1:8000 \
    config.wsgi:application
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Change the user, group, and paths to match your server. Configure environment variables securely using protected service configuration or an appropriately permissioned environment file.

Reload systemd:

```
sudo systemctl daemon-reload
```

Enable the service at boot:

```
sudo systemctl enable gunicorn
```

Start the service:

```
sudo systemctl start gunicorn
```

Restart:

```
sudo systemctl restart gunicorn
```

Check status:

```
sudo systemctl status gunicorn
```

Follow logs:

```
sudo journalctl -u gunicorn -f
```

View recent logs:

```
sudo journalctl -u gunicorn -n 100 --no-pager
```

---

18. HTTPS, Domain, and Firewall

Configure a domain

1. Register a domain with a domain provider.
2. Create an A record pointing to your VPS public IPv4 address.
3. Configure the `www` subdomain if needed.
4. Wait for DNS propagation.
5. Test the domain against your server.

Configure UFW

Make sure SSH access is allowed before enabling the firewall, or you may lock yourself out.

```
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status
```

If your server uses a different SSH port, allow that port instead.

Configure HTTPS

Certbot can obtain and renew certificates for Nginx. Follow the installation instructions appropriate for your Ubuntu version and installation method.
Test certificate renewal:

```
sudo certbot renew --dry-run
```

Confirm that automatic renewal is configured.

Reverse-proxy security

If Nginx terminates TLS, Django may need trusted-proxy configuration to recognize the original HTTPS scheme. Trust headers only from known proxies. Incorrect proxy trust can create security vulnerabilities.

---

19. Django Channels and WebSockets

What is Django Channels?

Django Channels extends Django to support protocols such as WebSockets. It is useful for:

- Chat applications
- Real-time notifications
- Live updates
- Other persistent connections

A Redis channel layer allows application processes to exchange messages.
Install:

```
pip install channels channels-redis
```

Add to `INSTALLED_APPS`:

```
INSTALLED_APPS = [
    # Existing apps
    "channels",
]
```

Configure in `settings.py`:

```
ASGI_APPLICATION = "config.asgi.application"

CHANNEL_LAYERS = {
    "default": {
        "BACKEND": "channels_redis.core.RedisChannelLayer",
```

```
        "CONFIG": {
            "hosts": ["redis://127.0.0.1:6379/4"],
        },
    },
}
```

This example uses Redis database `4` for the channel layer.

A complete Channels application also needs consumers, WebSocket routing, ASGI routing, and suitable authentication and authorization.

Deployment note: WebSockets require an ASGI-capable server and appropriate Nginx upgrade headers. A standard Gunicorn WSGI setup does not provide WebSocket support by itself.

---

20. Waitress and uWSGI

Waitress

Waitress is a WSGI server that can be used to serve Django applications, including in native Windows environments.

Install:

```
pip install waitress
```

Run:

```
waitress-serve --listen=127.0.0.1:8000 config.wsgi:application
```

Open:

```
http://127.0.0.1:8000/
```

Waitress is a WSGI server, not a replacement for an ASGI server when deploying Django Channels WebSockets.

uWSGI

uWSGI is another application server option for Django WSGI applications.

Install:

```
pip install uwsgi
```

Example command:

```
uwsgi --http 127.0.0.1:8000 \
      --module config.wsgi:application \
```

```
        --master \
        --processes 4 \
        --threads 2
```

Some environments may require a compiler or additional system libraries to install uWSGI.

Which server should you choose?

- **Gunicorn:** A common choice for Linux Django WSGI deployment.
- **uWSGI:** An alternative WSGI/application server.
- **Waitress:** Useful in native Windows environments.
- **ASGI server:** Needed for ASGI-specific requirements and WebSocket deployments.

You do not need to run all these servers in the same project.

---

21. Full Django + Redis + Celery Workflow

A common architecture:

```
Client / Browser
        |
        v
      Nginx
        |
        v
     Gunicorn
        |
        v
      Django
        |
        +------ PostgreSQL
        |
        +------ Redis Cache
        |
        +------ Redis Broker
                      |
                      v
                Celery Worker
                      |
                      v
              Background Tasks
```

For scheduled tasks:

```
Celery Beat
    |
    v
Redis Broker
    |
    v
Celery Worker
```

For WebSockets, add an ASGI server and Django Channels as required.

What does each component do?

Component	Purpose
Django	Application logic
DRF	REST APIs
PostgreSQL	Persistent relational data
Redis	Cache, broker, and channel layer
Celery	Background task execution
Celery Beat	Scheduling tasks

Gunicorn	WSGI application server
Nginx	Reverse proxy, static files, HTTPS
systemd	Service management
Channels	WebSockets and asynchronous protocol support
Waitress/uWSGI	Alternative WSGI server options

---

22. Local Development — Terminal by Terminal

These commands are for Linux/WSL2. Run them from your project root with the virtual environment activated where needed.

Terminal 1: Redis

```
sudo systemctl start redis-server
redis-cli ping
```

Expected:

```
PONG
```

Terminal 2: Django

```
cd ~/projects/django-deployment
source venv/bin/activate
python manage.py runserver
```

Terminal 3: Celery Worker

```
cd ~/projects/django-deployment
source venv/bin/activate
celery -A config worker --loglevel=info
```

Terminal 4: Celery Beat (optional)

```
cd ~/projects/django-deployment
source venv/bin/activate
celery -A config beat --loglevel=info
```

Production environment

Do not use Django's development server for public production traffic. Run Gunicorn or an appropriate ASGI server behind Nginx, and manage the application and Celery processes with systemd or another suitable process manager.

---

23. Monitoring and Debugging

Django checks

```
python manage.py check
python manage.py check --deploy
python manage.py showmigrations
```

Gunicorn

```
sudo systemctl status gunicorn
sudo journalctl -u gunicorn -n 100 --no-pager
```

Nginx

```
sudo nginx -t
sudo systemctl status nginx
sudo tail -f /var/log/nginx/error.log
sudo tail -f /var/log/nginx/access.log
```

Redis

```
redis-cli ping
sudo systemctl status redis-server
```

Celery

Inspect the worker logs. Verify that the worker is running, the broker URL is correct, and the task is registered and discoverable.

Listening ports

```
sudo ss -ltnp
```

Common errors

1. `ModuleNotFoundError`

Possible cause: The wrong virtual environment is active, or the package is missing.

Fix:

```
source venv/bin/activate
pip install -r requirements.txt
```

2. `DisallowedHost`

Cause: The request hostname is missing from `ALLOWED_HOSTS`.

Fix: Add the actual domain or hostname.

3. `502 Bad Gateway`

Possible causes:

- Gunicorn is stopped.
- Nginx's `proxy_pass` is incorrect.
- Gunicorn is listening on another port.
- Service startup or permission settings are wrong.

Check:

```
sudo systemctl status gunicorn
sudo journalctl -u gunicorn -n 100 --no-pager
sudo nginx -t
```

4. Static files are missing

Check:

```
python manage.py collectstatic
sudo nginx -t
```

Verify `STATIC_ROOT`, Nginx's `alias` path, and directory permissions.

5. Celery tasks are not executing

Check:

- Redis is running.
- The Celery worker is active.
- The project name `config` is correct.
- Tasks are being discovered.
- The broker URL is consistent across processes.

6. Database connection errors

Check:

- PostgreSQL is running.
- The database and user exist.
- Credentials are correct.
- The hostname and port are correct.
- The database user has the necessary privileges.

7. HTTPS redirect loop

Check:

- Nginx TLS termination.
- The `X-Forwarded-Proto` header.
- Django's trusted-proxy and secure-scheme configuration.
- Whether proxy headers are trusted only from known proxies.

---

24. Backup and Recovery

Production data should be backed up regularly.

PostgreSQL backup

```
pg_dump -h 127.0.0.1 -U django_user -d django_db \
    -Fc -f django_db_backup.dump
```

Prepare the database and required permissions before restoring:

```
pg_restore -h 127.0.0.1 -U django_user \
    -d django_db django_db_backup.dump
```

Store backups securely and test the restore procedure regularly.

What should be backed up?

- PostgreSQL database
- User-uploaded media
- Required application configuration
- Deployment and recovery instructions
- Other persistent files required by the application

Redis cache data can often be recreated. If Redis stores important persistent data, plan its persistence and backup strategy separately.

Protect secret backups using encrypted, access-controlled storage.

---

25. Production Deployment Checklist

Before deployment, verify the following:

- `DEBUG=False`
- Strong, private `SECRET_KEY`
- Correct `ALLOWED_HOSTS`
- Production database configured
- Environment variables stored securely
- Database migrations applied
- `collectstatic` completed
- `python manage.py check --deploy` reviewed
- Gunicorn or an appropriate ASGI server configured
- Nginx reverse proxy configured
- HTTPS certificate active
- Firewall and SSH security configured
- Redis protected from public access
- Celery worker running, if required
- Celery Beat running, if scheduled tasks are required
- Logging and monitoring configured
- Database backup and restore tested
- Media upload storage configured
- Dependencies recorded in `requirements.txt`
- Production smoke tests completed

---

26. Serial-Wise Learning Roadmap

Follow this order for learning and practice:

1. WSL2 and Ubuntu Setup
2. Linux Terminal Basics
3. Python Virtual Environment
4. Django Project Setup
5. Django Models, Views, URLs, and Admin
6. Django REST Framework
7. Authentication and Permissions
8. Git and GitHub
9. Redis Installation
10. Django Redis Cache
11. Django Sessions
12. Celery Setup
13. Celery Tasks
14. Celery Beat
15. PostgreSQL
16. Production Settings and Environment Variables
17. Gunicorn
18. Nginx
19. systemd Services
20. Domain and HTTPS
21. Django Channels and ASGI
22. Monitoring, Logs, and Backup
23. Waitress/uWSGI — optional alternatives

Recommended Practice Project

Start with a CRUD REST API. Then:

1. Connect PostgreSQL.
2. Add API validation and permissions.
3. Add Redis caching.
4. Create a background email or report task using Celery.
5. Serve the WSGI application with Gunicorn.
6. Configure Nginx as a reverse proxy.

7. Create systemd services.
8. Configure a domain and HTTPS.
9. Practice logging, backup, and recovery.
10. Add Channels/WebSockets if the application requires them.

Final Summary

The goal of Django deployment is to run an application securely, reliably, and maintainably in production.

A common WSGI deployment architecture:

```
Internet
   |
   v
Nginx + HTTPS
   |
   v
Gunicorn
   |
   v
Django + DRF
   |
   +---- PostgreSQL
   |
   +---- Redis Cache
   |
   +---- Celery Broker --> Celery Worker
```

Not every application needs Redis, Celery, Channels, Waitress, and uWSGI at the same time. Choose technologies based on the application's requirements.

Recommended starting stack: Django + DRF + PostgreSQL + Gunicorn + Nginx. Add Redis and Celery when caching or background task processing is needed.